Lab 03 — Portfolio and Budget Tracker

Follow a basket of companies, or a budget, and understand what moved and why.

Introduction

This project has the richest data of the three, which makes it the most tempting to overbuild. The table that tries to show everything ends up showing nothing. The table that picks one question and answers it well ends up with something worth demoing.

> ⚠️ This is a **tracking and explaining** tool, not an advice tool. It doesn't tell anyone what to buy or sell, and it never connects to a real brokerage account or real money. The data you'll use is simulated.

Before You Start

- **One laptop, five people.** Decide who drives. The driver types; everyone else plans, watches the output and proposes what to try next. Swap if you like.
- **Your table's job is not to finish.** It's to get something real running and understand how you got there.
- **Laptops stay closed** until Step 02. The first thirty minutes are paper and pens — and that's the part that determines whether the build works.

What You'll Build

A working page that follows a set of holdings — or a budget — and:

- Shows what changed.
- Explains *why*, in plain language, using news and context rather than numbers alone.
- **Does something** with that, instead of waiting to be looked at.

Step 01: Brainstorm Together — Laptops Closed

About 30 minutes. Paper and pens on the table.

Write your answers down. At the end of this step you'll hand what you wrote to Claude, so write it for a reader.

Portfolio or Budget? And for Whom?

- Decide which you're building. They share a shape — things with values that change over time — but the user is different.
- Who is this person? A student with their first few hundred dollars invested, or someone tracking spending across a semester?
- What question do they open it to answer? Write that question down in one sentence. Everything else serves it.

What Does the Screen Show?

- Sketch it. One page, boxes and labels.
- One big number at the top, or a list, or a chart? Decide and justify it.
- Name two things you're deliberately not building. "Everything a real brokerage app has" is the trap here.

Where Does the Data Come From?

- You will be given a set of simulated holdings, prices and news to work with.

[PLACEHOLDER - how students connect to the workshop MCP server, once the server exists]

- What's in the data? Which parts does your one question actually need?

What Does It Do, Not Just Show?

This is the agentic question, and the one worth most of your time.

- A page that **displays** a portfolio is a screenshot. What does yours **do**?
- Some directions worth arguing about:
    - It explains the day's biggest move in a sentence, by connecting a price change to a news item.
    - It flags when one holding has grown into too large a share of the whole.
    - It runs each morning and writes a short summary of what changed overnight.

    - It answers a typed question about the portfolio by going and looking rather than guessing.
- Pick **one**. Write down what triggers it, and what the result is.

Step 02: Turn Your Notes into a Prompt

Now the laptop opens.

- **Open a Cowork session** connected to your project folder from Lab 00.
- **Two ways to get your plan in**, and the second is usually better:
    - Type it out.
    - **Photograph your notes** with a phone and attach the picture. Claude reads handwriting and sketches, and your sketch of the screen is usually clearer than a paragraph describing it.
- Write the prompt in your own words: who it's for, what the screen shows, where the data comes from, what it does.

Step 03: Let Claude Interview You

Optional, and recommended. Five minutes that saves twenty.

> **PROMPT**
>
> `Before you build anything, interview me. Ask me five multiple-choice questions about what I want, one at a time. Then write a short plan as MVP Plan.md and show me a simple mockup with sample data. Wait for me to approve it before you write any other code.`

This catches what your table forgot. It is also the lecture in miniature: you are directing an agent, not placing an order.

Step 04: Build It

- Send your prompt.

- Claude will ask permission before it does things. **Click Allow** — or **Allow for this task** to reduce the clicking.

Step 05: While You Wait

Builds take **ten minutes or more**. Plan for it.

- **Stay near the laptop.** Claude pauses to ask questions and waits until someone answers.
- **Non-drivers, you have work to do.** Discuss:
    - *Why is Markdown a useful language for configuring agents?*
    - Your app will explain *why* something moved. How would you know if that explanation were wrong?
    - Where's the line between a tool that explains and a tool that advises — and why does it matter here?
- **Compare with your neighbors.**

Step 06: Run It and Try It

- Claude will tell you how to start your app and give you a web address to open.
- Ask it the one question you wrote down in Step 01. Does it answer it?
- Find one thing that's wrong or missing. That's your next prompt.

Step 07: Add Your Own Twist

Whatever time is left is yours.

If You Get Stuck

- **Strange file errors, or the build keeps failing:** check your project folder isn't in iCloud, Dropbox, OneDrive, Box or Google Drive. Move it to your home folder and try again.

- **Claude seems frozen:** look for a permission prompt waiting for a click.
- **The app won't start:** paste the error message back to Claude and ask it to fix it. This is normal.
- **You're out of ideas:** see the Answer Sheet below.

Key Takeaways

- The thirty minutes on paper did more for your build than any single prompt.
- You directed an agent. It planned, used tools and acted — you decided what mattered.
- The gap between "displays information" and "does something" is the gap between an app and an agent.
- An explanation produced by a model is a claim, not a fact. Knowing how you'd check it is part of building responsibly.

Answer Sheet

Use these if your table is stuck. Reading them first spoils the exercise.

[PLACEHOLDER - suggested prompts: a starting prompt, a prompt for adding the agentic action, and two recovery prompts for common failures]